

PREMIUM CONTRACT: СОВМЕСТНЫЕ ВЛОЖЕНИЯ (COLLABORATIVE)

Версия: 1.0.0

Дата: 2026-06-14

Статус: ☒ Complete

Доступ:  Только "Шейх"

1. ОБЗОР

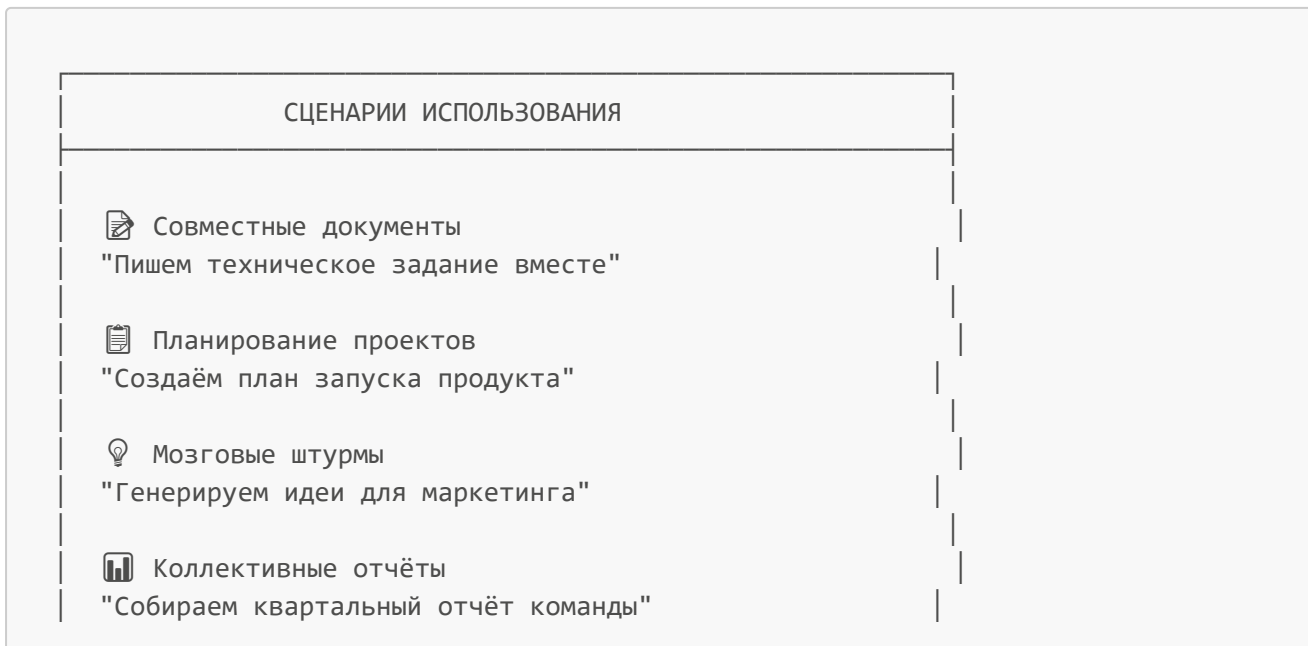
1.1 Назначение

Совместные вложения (Collaborative) — это premium вложение для создания контента, который можно редактировать совместно в реальном времени прямо в чате Balloo Messenger. Идеально для мозговых штурмов, планирования и совместной работы.

1.2 Статус доступа

Параметр	Значение
Доступ	 Premium (Шейх)
Категория	Продуктивность / Collaboration
Сложность	 Высокая
Время реализации	5-7 дней
Real-time	Да (WebSocket + OT/CRDT)

1.3 Use Cases



 Голосования и решения
"Принимаем решение по бюджету"

2. ТИПЫ ДАННЫХ

2.1 Основной интерфейс

```
/**
 * Тип вложения
 */
export type CollaborativeAttachmentType = 'collaborative';

/**
 * Типы совместных вложений
 */
export type CollaborativeType =
  | 'document'      // Текстовый документ
  | 'whiteboard'    // Виртуальная доска
  | 'spreadsheet'   // Таблица
  | 'list'          // Расширенный список
  | 'mindmap'       // Интеллект-карта
  | 'kanban'        // Канбан-доска
  | 'code'          // Код (с подсветкой)
  | 'form';         // Форма

/**
 * Статусы редактирования
 */
export type EditStatus =
  | 'idle'          // Никто не редактирует
  | 'editing'       // Кто-то редактирует
  | 'conflict';     // Конфликт версий

/**
 * Совместное вложение
 */
export interface CollaborativeAttachment {
  type: 'collaborative';
  attachmentId: string;
  collabId: string;

  // Тип
  collabType: CollaborativeType;

  // Контент
  title: string;
```

```

description?: string;
content: CollaborativeContent;

// Участники
owner: CollaborativeUser;
editors: CollaborativeUser[];
viewers: CollaborativeUser[];
maxEditors: number;

// Права доступа
permissions: CollaborativePermissions;

// Real-time
realtime: RealtimeConfig;
activeEditors: string[]; // userIds сейчас редактируют
cursors: EditorCursor[]; // Позиции курсоров

// Версионирование
version: number;
versionHistory: VersionEntry[];
lastSavedAt: number;

// Блокировки
locks: Record<string, string>; // section -> userId

// Статус
status: 'active' | 'archived' | 'deleted';

// Метаданные
metadata: Record<string, any>;

createdAt: number;
updatedAt: number;
}

/**
 * Пользователь в совместном вложении
 */
export interface CollaborativeUser {
  userId: string;
  displayName: string;
  avatar?: string;
  isSheikh: boolean;
  role: 'owner' | 'editor' | 'viewer';
  permissions: PermissionLevel;
  joinedAt: number;
  lastActiveAt?: number;
  color: string; // Цвет курсора
  isOnline: boolean;
}

export type PermissionLevel = 'read' | 'write' | 'admin';

```

```

/**
 * Контент (абстрактный, зависит от типа)
 */
export interface CollaborativeContent {
  type: CollaborativeType;
  data: any;
  schema: ContentSchema;
}

/**
 * Документ
 */
export interface DocumentContent extends CollaborativeContent {
  type: 'document';
  data: {
    blocks: DocumentBlock[];
    styles: Record<string, any>;
  };
}

export interface DocumentBlock {
  id: string;
  type: 'paragraph' | 'heading' | 'list' | 'quote' | 'code' | 'image';
  content: string;
  children?: DocumentBlock[];
  styles?: Record<string, any>;
}

/**
 * Виртуальная доска
 */
export interface WhiteboardContent extends CollaborativeContent {
  type: 'whiteboard';
  data: {
    elements: WhiteboardElement[];
    viewport: { x: number; y: number; zoom: number };
  };
}

export interface WhiteboardElement {
  id: string;
  type: 'rectangle' | 'circle' | 'line' | 'arrow' | 'text' | 'sticky' |
'drawing';
  x: number;
  y: number;
  width?: number;
  height?: number;
  content?: string;
  style: {
    color: string;
    strokeWidth: number;
    fontSize?: number;
  };
};

```

```

    createdBy: string;
}

/**
 * Канбан-доска
 */
export interface KanbanContent extends CollaborativeContent {
    type: 'kanban';
    data: {
        columns: KanbanColumn[];
    };
}

export interface KanbanColumn {
    id: string;
    title: string;
    cards: KanbanCard[];
    order: number;
}

export interface KanbanCard {
    id: string;
    title: string;
    description?: string;
    assignees?: string[];
    dueDate?: number;
    priority: 'low' | 'medium' | 'high';
    tags?: string[];
    order: number;
}

/**
 * Курсор редактора
 */
export interface EditorCursor {
    userId: string;
    userName: string;
    color: string;
    position: {
        blockId?: string;
        offset: number;
        x?: number;
        y?: number;
    };
    selection?: {
        start: number;
        end: number;
    };
    lastUpdate: number;
}

/**
 * Конфигурация real-time

```

```

*/
export interface RealtimeConfig {
  enabled: boolean;
  protocol: 'websocket' | 'webrtc';
  syncInterval: number; // ms
  conflictResolution: 'ot' | 'crdt' | 'last_write_wins';
  offlineSupport: boolean;
  maxConcurrentEditors: number;
}

/**
 * Запись версии
 */
export interface VersionEntry {
  version: number;
  editedBy: string;
  editedAt: number;
  changes: ChangeSummary;
  snapshot?: string; // URL snapshot
  canRestore: boolean;
}

export interface ChangeSummary {
  added: number;
  deleted: number;
  modified: number;
  description?: string;
}

/**
 * Права доступа
 */
export interface CollaborativePermissions {
  canEdit: boolean;
  canComment: boolean;
  canShare: boolean;
  canDelete: boolean;
  canExport: boolean;
  canInvite: boolean;
  requireApproval: boolean;
  allowAnonymous: boolean;
}

```

2.2 Операции редактирования

```

/**
 * Операция (для Operational Transformation)
 */
export interface Operation {
  id: string;
}

```

```

    type: OperationType;
    userId: string;
    timestamp: number;
    data: any;
    version: number;
}

export type OperationType =
    | 'insert'
    | 'delete'
    | 'update'
    | 'move'
    | 'format';

export interface InsertOperation extends Operation {
    type: 'insert';
    data: {
        position: number;
        content: string | any;
        blockId?: string;
    };
}

export interface DeleteOperation extends Operation {
    type: 'delete';
    data: {
        position: number;
        length: number;
        blockId?: string;
    };
}

export interface UpdateOperation extends Operation {
    type: 'update';
    data: {
        blockId: string;
        field: string;
        oldValue: any;
        newValue: any;
    };
}

/**
 * CRDT (Conflict-free Replicated Data Type)
 */
export interface CRDTValue {
    id: string;
    value: any;
    timestamp: number;
    siteId: string;
    counter: number;
}

```

3. API SPECIFICATION

3.1 Endpoints

COLLABORATIVE API ENDPOINTS		
POST	/api/v1/collab/create	Создать
POST	/api/v1/collab/:id/join	Присоединиться
POST	/api/v1/collab/:id/leave	Покинуть
POST	/api/v1/collab/:id/invite	Пригласить
POST	/api/v1/collab/:id/permissions	Права доступа
GET	/api/v1/collab/:id	Получить
GET	/api/v1/collab/:id/history	История версий
POST	/api/v1/collab/:id/restore	Восстановить
GET	/api/v1/collab/:id/export	Экспорт
DELETE	/api/v1/collab/:id/archive	Архивировать
WS	/ws/collab/:id	WebSocket

3.2 Создать совместное вложение

Endpoint: POST /api/v1/collab/create

Headers:

```
Authorization: Bearer <token>
Content-Type: application/json
X-Sheikh-Status: true
```

Request Body:

```
{
  "collabType": "document",
  "title": "ТЗ проекта Balloo 2.0",
  "description": "Совместная работа над техническим заданием",
  "content": {
    "type": "document",
    "data": {
      "blocks": [
        {
          "id": "block_1",
          "type": "heading",
```



```

        "content": "Введение"
    },
    {
        "id": "block_2",
        "type": "paragraph",
        "content": ""
    }
]
},
"editors": ["user_123", "user_456"],
"permissions": {
    "canEdit": true,
    "canComment": true,
    "canShare": false,
    "requireApproval": false
},
"realtime": {
    "enabled": true,
    "maxConcurrentEditors": 10
}
}

```

Response (Success 201):

```

{
    "success": true,
    "data": {
        "collabId": "collab_abc123",
        "attachmentId": "att_xyz789",
        "inviteUrl": "https://balloo.app/collab/join/collab_abc123",
        "status": "active",
        "version": 1,
        "editors": [
            {
                "userId": "user_current",
                "displayName": "Вы",
                "role": "owner"
            }
        ],
        "websocketUrl": "wss://balloo.app/ws/collab/collab_abc123"
    }
}

```

3.3 WebSocket события

```

// Клиент → Сервер
type ClientEvent =

```

```

| { type: 'join'; userId: string; token: string }
| { type: 'leave' }
| { type: 'operation'; operation: Operation }
| { type: 'cursor_update'; cursor: EditorCursor }
| { type: 'lock_request'; section: string }
| { type: 'lock_release'; section: string }
| { type: 'save_request' };

// Сервер → Клиент
type ServerEvent =
| { type: 'joined'; users: CollaborativeUser[] }
| { type: 'user_joined'; user: CollaborativeUser }
| { type: 'user_left'; userId: string }
| { type: 'operation'; operation: Operation }
| { type: 'operations'; operations: Operation[] } // Batch
| { type: 'cursor_update'; cursor: EditorCursor }
| { type: 'cursors'; cursors: EditorCursor[] }
| { type: 'lock_granted'; section: string }
| { type: 'lock_denied'; section: string; reason: string }
| { type: 'saved'; version: number }
| { type: 'conflict'; operations: Operation[] }
| { type: 'error'; error: CollaborativeError };

```

3.4 История версий

Endpoint: `GET /api/v1/collab/:id/history`

Query Parameters:

```

?limit=50
&offset=0
&from=1717200000000
&to=1718400000000

```

Response:

```

{
  "success": true,
  "data": {
    "collabId": "collab_abc123",
    "currentVersion": 47,
    "versions": [
      {
        "version": 47,
        "editedBy": {
          "userId": "user_123",
          "displayName": "Иван Петров"
        }
      },
    ]
  }
}

```

```

    "editedAt": 1718400000000,
    "changes": {
      "added": 150,
      "deleted": 20,
      "modified": 5,
      "description": "Добавил раздел API"
    },
    "canRestore": true
  },
  {
    "version": 46,
    "editedBy": {
      "userId": "user_456",
      "displayName": "Мария Сидорова"
    },
    "editedAt": 1718399000000,
    "changes": {
      "added": 300,
      "deleted": 50,
      "modified": 10,
      "description": "Расширил введение"
    },
    "canRestore": true
  }
],
"pagination": {
  "total": 47,
  "limit": 50,
  "offset": 0,
  "hasMore": false
}
}
}

```

4. БИЗНЕС-ЛОГИКА

4.1 Operational Transformation (OT)

```

/**
 * OT Engine для совместного редактирования
 */
class OTEngine {

  // Трансформация операции против другой операции
  transform(op1: Operation, op2: Operation): Operation {
    if (op1.type === 'insert' && op2.type === 'insert') {
      return this.transformInsertInsert(op1, op2);
    }
    if (op1.type === 'insert' && op2.type === 'delete') {

```

```

        return this.transformInsertDelete(op1, op2);
    }
    if (op1.type === 'delete' && op2.type === 'insert') {
        return this.transformDeleteInsert(op1, op2);
    }
    if (op1.type === 'delete' && op2.type === 'delete') {
        return this.transformDeleteDelete(op1, op2);
    }
    return op1;
}

private transformInsertInsert(op1: InsertOperation, op2: InsertOperation):
InsertOperation {
    // Если op2 вставила перед op1, сдвигаем позицию op1
    if (op2.data.position <= op1.data.position) {
        return {
            ...op1,
            data: {
                ...op1.data,
                position: op1.data.position + this.getContentLength(op2.data.content)
            }
        };
    }
    return op1;
}

private transformInsertDelete(op1: InsertOperation, op2: DeleteOperation):
InsertOperation {
    // Если op2 удалила перед op1, сдвигаем позицию op1
    if (op2.data.position < op1.data.position) {
        return {
            ...op1,
            data: {
                ...op1.data,
                position: Math.max(
                    op2.data.position,
                    op1.data.position - op2.data.length
                )
            }
        };
    }
    return op1;
}

private transformDeleteInsert(op1: DeleteOperation, op2: InsertOperation):
DeleteOperation {
    // Если op2 вставила внутри удаляемого диапазона
    if (op2.data.position >= op1.data.position &&
        op2.data.position < op1.data.position + op1.data.length) {
        return {
            ...op1,
            data: {
                ...op1.data,

```

```

        length: op1.data.length + this.getContentLength(op2.data.content)
    }
    };
}
// Если op2 вставила перед удаляемым диапазоном
if (op2.data.position < op1.data.position) {
    return {
        ...op1,
        data: {
            ...op1.data,
            position: op1.data.position + this.getContentLength(op2.data.content)
        }
    };
}
return op1;
}

private transformDeleteDelete(op1: DeleteOperation, op2: DeleteOperation):
DeleteOperation {
    // Сложная логика для пересекающихся удалений
    if (op2.data.position >= op1.data.position + op1.data.length) {
        return op1; // Нет пересечения
    }
    if (op2.data.position + op2.data.length <= op1.data.position) {
        return {
            ...op1,
            data: {
                ...op1.data,
                position: op1.data.position - op2.data.length
            }
        };
    }
    // Пересечение – сложная логика
    return this.handleOverlappingDeletes(op1, op2);
}

private getContentLength(content: any): number {
    if (typeof content === 'string') return content.length;
    return 1; // Для блоков
}
}

```

4.2 Обработка операций

```

async function handleOperation(
    collabId: string,
    operation: Operation
): Promise<OperationResult> {

    const collab = await getCollaborativeAttachment(collabId);

```

```

// 1. Проверка прав
const user = collab.editors.find(e => e.userId === operation.userId);
if (!user || user.permissions !== 'write') {
    throw new PermissionDeniedError();
}

// 2. Проверка блокировки
const lock = getSectionLock(operation.data.blockId);
if (lock && lock !== operation.userId) {
    throw new SectionLockedError(lock);
}

// 3. Трансформация операции
const pendingOps = getPendingOperations(collabId);
let transformedOp = operation;

for (const pendingOp of pendingOps) {
    if (pendingOp.userId !== operation.userId) {
        transformedOp = otEngine.transform(transformedOp, pendingOp);
    }
}

// 4. Применение операции
await applyOperation(collabId, transformedOp);

// 5. Сохранение в историю
await saveOperation(collabId, transformedOp);

// 6. Отправка другим редакторам
broadcastToEditors(collabId, {
    type: 'operation',
    operation: transformedOp
});

// 7. Автосохранение версии
if (shouldSaveVersion(collabId)) {
    await saveVersion(collabId);
}

return { success: true, operation: transformedOp };
}

```

4.3 Блокировки секций

```

interface SectionLock {
    sectionId: string;
    userId: string;
    acquiredAt: number;
    expiresAt: number;
}

```

```

}

async function acquireLock(
  collabId: string,
  sectionId: string,
  userId: string
): Promise<LockResult> {

  const existingLock = await getLock(collabId, sectionId);

  if (existingLock) {
    if (existingLock.userId === userId) {
      // Продлить свою блокировку
      existingLock.expiresAt = Date.now() + 30000; // 30 сек
      await updateLock(existingLock);
      return { acquired: true, lock: existingLock };
    }

    if (existingLock.expiresAt > Date.now()) {
      // Блокировка активна другим пользователем
      return {
        acquired: false,
        reason: 'locked_by_other',
        lockedBy: existingLock.userId,
        expiresAt: existingLock.expiresAt
      };
    }

    // Блокировка истекла
    await releaseLock(collabId, sectionId);
  }

  // Создать новую блокировку
  const lock: SectionLock = {
    sectionId,
    userId,
    acquiredAt: Date.now(),
    expiresAt: Date.now() + 30000
  };

  await createLock(lock);

  // Heartbeat для продления
  startLockHeartbeat(collabId, sectionId, userId);

  return { acquired: true, lock };
}

async function releaseLock(
  collabId: string,
  sectionId: string,
  userId?: string
): Promise<void> {

```

```

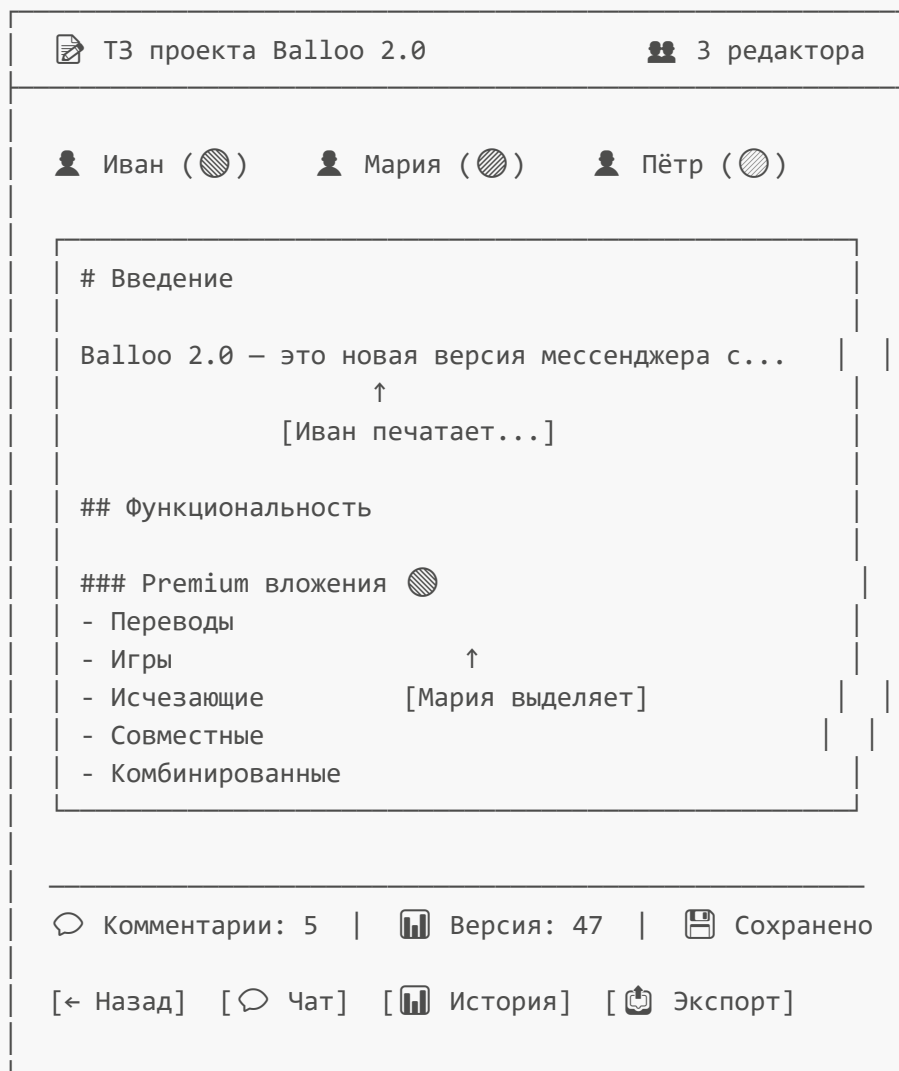
await deleteLock(collabId, sectionId);

broadcastToEditors(collabId, {
  type: 'lock_released',
  sectionId
});
}

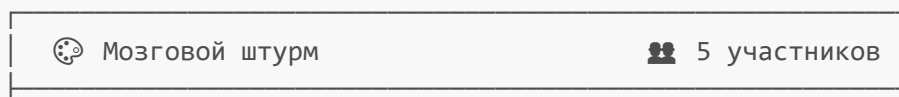
```

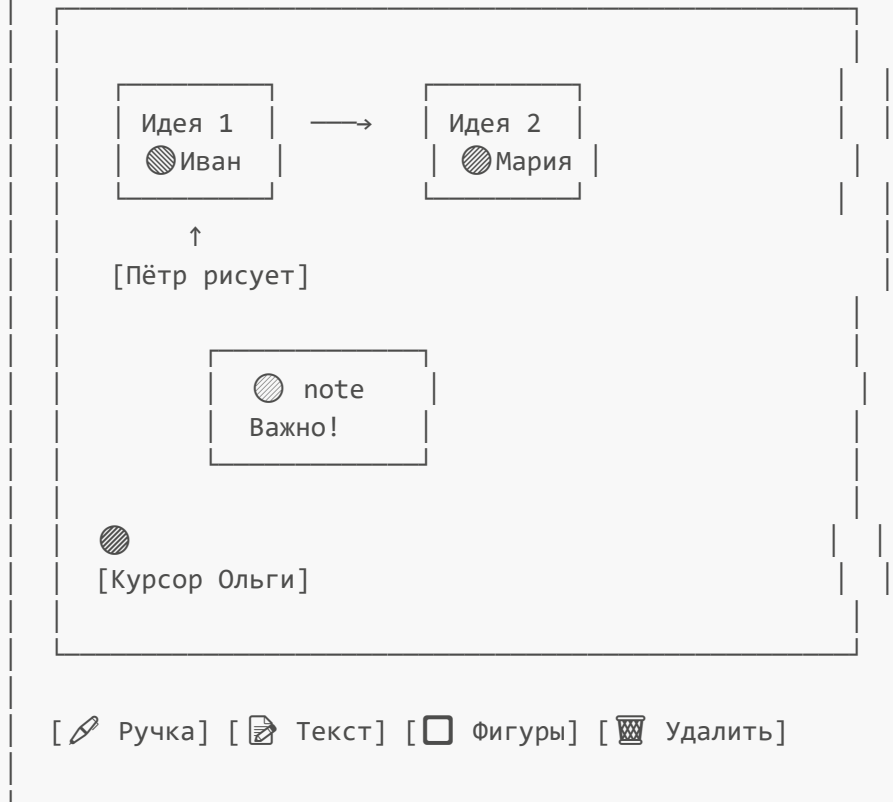
5. UI/UX SPECIFICATION

5.1 Совместный документ

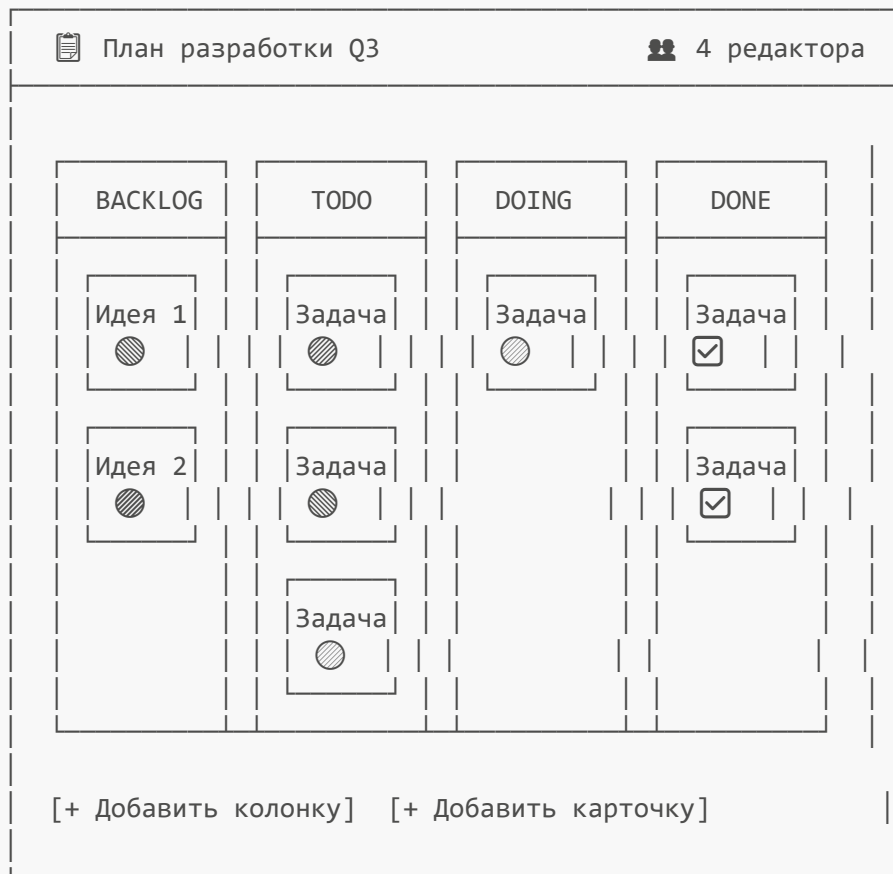


5.2 Виртуальная доска





5.3 Канбан-доска



5.4 Модальное окно (не Шейх)

💎 Совместная работа только для Шейх

Создавайте и редактируйте документы вместе в реальном времени!

 Совместные документы

 Виртуальные доски

 Канбан-доски

 Интеллект-карты

☒ Real-time синхронизация

☒ История версий

☒ Комментарии и обсуждения

☒ Экспорт в PDF/DOCX

[✕ Отмена]

[📄 Оформить Шейх за \$9.99/мес]

6. МЕТРИКИ

```
interface CollaborativeMetrics {  
  // Создание  
  totalCollabs: number;  
  createdToday: number;  
  byType: Record<CollaborativeType, number>;  
  
  // Активность  
  activeCollabs: number;  
  concurrentEditors: number;  
  peakConcurrent: number;  
  
  // Вовлечённость  
  averageEditors: number;  
  averageSessionDuration: number;  
  operationsPerSecond: number;  
  
  // Контент  
  totalBlocks: number;  
  totalVersions: number;  
  averageVersionsPerCollab: number;
```

```
// Конфликты
conflictRate: number;
autoResolved: number;
manualResolved: number;
}
```

 **Статус документа:** Complete

 **Baloo - Проверни общение!**